DATA ANALYTICS PROJECT REPORT

CREDENTIALS

Name: Gunjan Ahuja
Course: B.tech 2nd year, (AI-DS)
Enrollment Number: 02025511924

INTRODUCTION

- **DATASET USED:** e-commerce dataset (Olist dataset)
- **GOAL:** This project focuses on analyzing delivery performance in an e-commerce system, with a specific emphasis on identifying the causes of delivery delays and understanding their impact on customers.
- **MAIN FOCUS:** Identifying factors affecting delays and customer impact
- **TOOLS USED:** Python, Pandas, Seaborn, Matplotlib, Google Colab

CODES AND INSIGHTS

STEP 1: Importing the libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

STEP 2 : Loading the datasets

```
orders = pd.read_csv("olist_orders_dataset.csv")
customers = pd.read_csv("olist_customers_dataset.csv")
order_items = pd.read_csv("olist_order_items_dataset.csv")
products = pd.read_csv("olist_products_dataset.csv")
payments = pd.read_csv("olist_order_payments_dataset.csv")
```

Insights: Only relevant datasets were selected to focus on delivery-related factors and avoid unnecessary complexity. Other datasets such as reviews, sellers, and geolocation were not included as they were not directly relevant to the objective of analyzing delivery delays and would increase complexity without contributing significant insights.

STEP 3: Understanding the data

3.1 Orders Dataset

```
orders.head()
orders.info()
orders.describe()
```

Insights: The orders dataset contains information about each order, including order status and multiple timestamps such as purchase date, delivery date, and estimated delivery date.

- The dataset contains around **99,000 orders**
- All columns are initially stored as **object type**, including dates (needs conversion later)
- Some columns like **order_delivered_carrier_date** have missing values
- Most orders have status = **delivered**, meaning analysis can focus on completed deliveries

3.2 Customers Dataset

```
customers.head()
customers.info()
customers.describe()
```

Insights: This dataset contains customer-related information such as location and unique identifiers.

- Each customer has a unique ID
- Location data includes **city and state**
- No missing values in customer data
- Can be used to analyze **geographical impact on delivery**

3.3 Order Items Dataset

```
order_items.head()
order_items.info()
order_items.describe()
```

Insights: This dataset provides details about products in each order, including price and freight value.

- Contains more rows than orders → multiple items per order
- Includes **price and shipping cost**
- Useful for analyzing impact of **order value on delays.**
- **Multiple items per order indicates one-to-many relationship.**

3.4 Products Dataset

```
products.head()
products.info()
products.describe()
```

Insights: Contains product-related details such as category, dimensions, and weight.

- Includes product size and weight
- Some missing values in product description-related columns
- Useful for analyzing whether **heavier/larger products cause delays**

3.5 Payments Dataset

```
payments.head()
payments.info()
payments.describe(include="all")
```

Insights: Contains payment-related information including payment type, installments, and total payment value.

- Multiple payment entries per order (installments)
- Payment types include credit card, debit card, etc.
- Useful to analyze if **payment method affects delivery delay.**

From the initial exploration, it is clear that the dataset contains multiple related tables connected through order_id and customer_id. Some datasets have one-to-many relationships (like order items and payments), which will increase the number of rows after merging. Missing values are present in delivery-related columns and will be handled in the data cleaning step. These datasets together allow analysis of delivery performance, delays, and influencing factors such as price, location, and payment method.

STEP 4: Merging the datasets

```
df = orders.merge(customers, on="customer_id", how="left")
df = df.merge(order_items, on="order_id" , how="left")
df = df.merge(payments, on="order_id", how="left")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
df['order_id'].nunique()
```

Insights:

- The final merged dataset contains:
  - **118,434 rows**
  - **22 columns**
- The total number of unique orders is:
  - **99,441**
- The number of rows increased after merging because some datasets, such as **order_items** and **payments**, contain multiple records for a single order.

This indicates a one-to-many relationship, where a single order can have multiple products and multiple payment records. As a result, the same order appears multiple times in the merged dataset.

I used left join instead of an inner join to avoid losing any order data. Using an inner join could remove orders that do not have corresponding entries in other tables, which would lead to incomplete analysis.

Hence, the merged dataset provides a unified view of each order by combining customer details, product information, and payment data. This enables deeper analysis of delivery performance and helps in identifying factors that contribute to delivery delays.

STEP 5: Data Cleaning

```
df = df[df['order_status'] == 'delivered']

df = df.dropna(subset=[
    'order_delivered_customer_date',
    'order_estimated_delivery_date'
])

df['payment_value'] =
df['payment_value'].fillna(df['payment_value'].median())
df['payment_type'] =
df['payment_type'].fillna(df['payment_type'].mode()[0])
cols = [
    'order_purchase_timestamp',
```

```
    'order_delivered_customer_date',
    'order_estimated_delivery_date'
]

for col in cols:
    df[col] = pd.to_datetime(df[col])
print(df.isnull().sum())
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

Insights:

1. Filtering Relevant Data

- Only delivered orders were retained
- This ensures delay calculations are meaningful
  (undelivered orders don't have delivery time so they are useless for analysis)

2. Handling Missing Values

- Rows missing delivery dates were removed
  (because delay cannot be calculated without them)
- Missing values in:
  - payment_value → filled with median
  - payment_type → filled with mode

Reasoning:

- Median avoids distortion from extreme values
- Mode keeps categorical consistency

3. Data Type Conversion

- Date columns were converted from object → datetime
- This allows time-based calculations like:
  - delivery time
  - delay

Because without conversion we can't do feature engineering.

4. Final Data Quality Check

- Remaining missing values are almost zero (very minimal)
- Final dataset contains:

- 115,030 rows
- 22 columns

NOTE: A very small number of missing values still remain in some non-critical columns (such as payment installments), which do not significantly impact the analysis.

STEP 6: Data Transformation

```
df['delivery_time'] = (df['order_delivered_customer_date'] -
df['order_purchase_timestamp']).dt.days
df['delay'] = (df['order_delivered_customer_date'] -
df['order_estimated_delivery_date']).dt.days
df['is_delayed'] = df['delay'] > 0


df['order_month'] = df['order_purchase_timestamp'].dt.month
df['order_day'] = df['order_purchase_timestamp'].dt.day_name()
df[['delivery_time','delay']].describe()
```

Insights: In this step, new features were created from existing columns to make the dataset more suitable for analysis. This process helps in extracting meaningful information from raw data and enables deeper insights into delivery performance**.**

- **Delivery Time:**
  A new column delivery_time was created by calculating the difference between the delivery date and purchase date. This represents the total time taken to deliver an order.
- **Delay:**
  A new column delay was created by calculating the difference between the actual delivery date and the estimated delivery date. This indicates whether the delivery was early or late.
- **Delay Indicator:**
  A boolean column is_delayed was created to classify orders as delayed (True) or not delayed (False), making it easier to analyze delay patterns.
- **Time-based Features:**
  `order_month` was extracted to analyze monthly trends.
  `order_day` was extracted to analyze patterns based on the day of the week.

Key Observations:

- The average delivery time is approximately **12 days**.
- The average delay is negative, indicating that most orders are delivered before the estimated date.
- Delivery time varies significantly, with some extreme cases present.

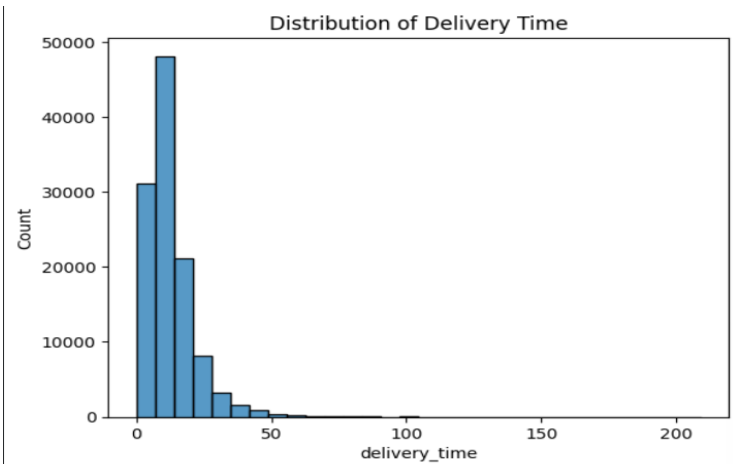
- Both early and delayed deliveries exist in the dataset.
- Time-based features enable analysis of seasonal and weekday trends.

Hence, These transformed features provide the foundation for further analysis by enabling the study of delivery efficiency, identifying delays, and understanding patterns based on time and other influencing factors.

STEP 7: Univariate Analysis

1.Distribution of Delivery Time

```
sns.histplot(df['delivery_time'], bins=30)
plt.title("Distribution of Delivery Time")
plt.show()
```

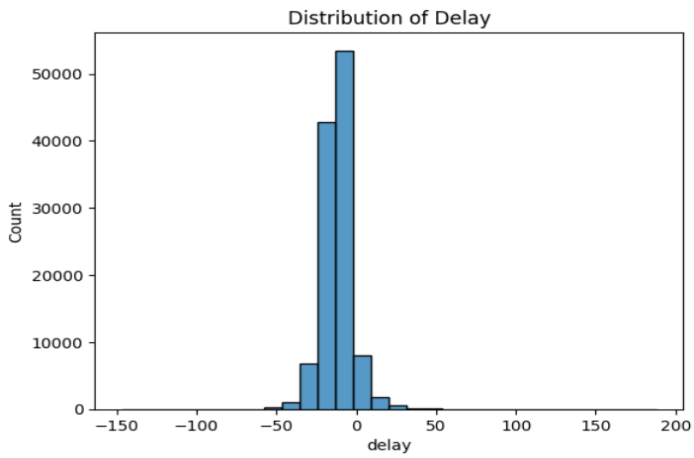


Insights:

- The distribution of **delivery time** is right-skewed, indicating that most orders are delivered within a shorter time period, while a few orders take significantly longer.
- The majority of deliveries are completed within approximately **0 to 20 days**.
- A small number of orders have very high delivery times, indicating the presence of outliers.

2.Distribution of Delay

```
sns.histplot(df['delay'], bins=30)
plt.title("Distribution of Delay")
plt.show()
```



Insights:

- The distribution of **delay** is centered around negative values.
- Most delay values are below zero, indicating that a large proportion of orders are delivered **before the estimated delivery date**.
- There are fewer cases of positive delay, meaning delayed deliveries are relatively less frequent.
- The delay distribution also shows some extreme values on both ends, indicating both very early and very late deliveries.

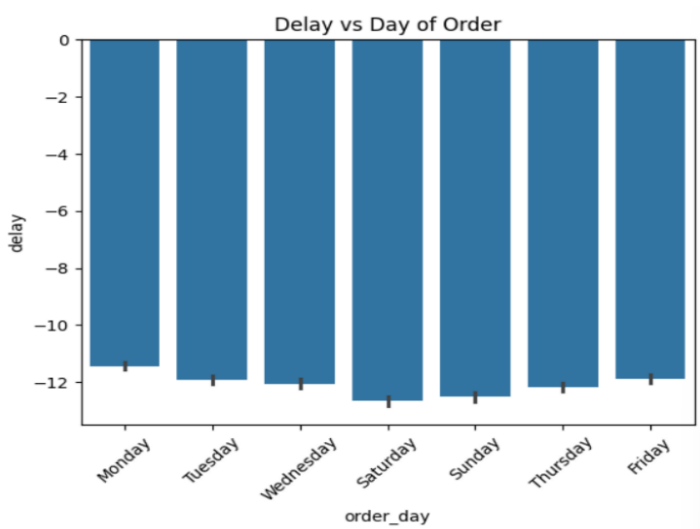
The univariate analysis shows that delivery performance is generally efficient, with most orders delivered within a reasonable time and often earlier than expected. However, the presence of outliers suggests that certain orders experience unusually long delivery times or significant delays, which require further analysis.

STEP 8: Bivariate Analysis

1.Delay vs Day of order

```
df.groupby('order_day')['delay'].mean().sort_values()

sns.barplot(x='order_day', y='delay', data=df)
plt.title("Delay vs Day of Order")
plt.xticks(rotation=45)
plt.show()
```



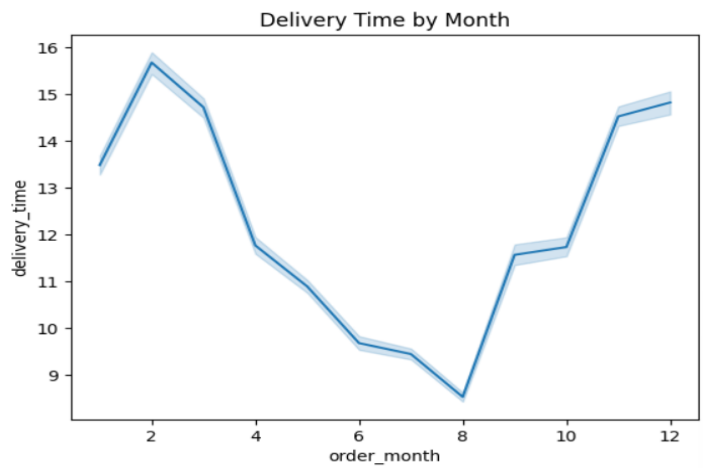
Insights:

- The average delay is negative for all days, indicating most orders are delivered before the estimated date.
- Slight variations exist across days, with some days showing relatively higher delays, but no drastic difference

is observed.

2.Delivery Time vs Month

```
df.groupby('order_month')['delivery_time'].mean()
sns.lineplot(x='order_month', y='delivery_time', data=df)
plt.title("Delivery Time by Month")
plt.show()
```

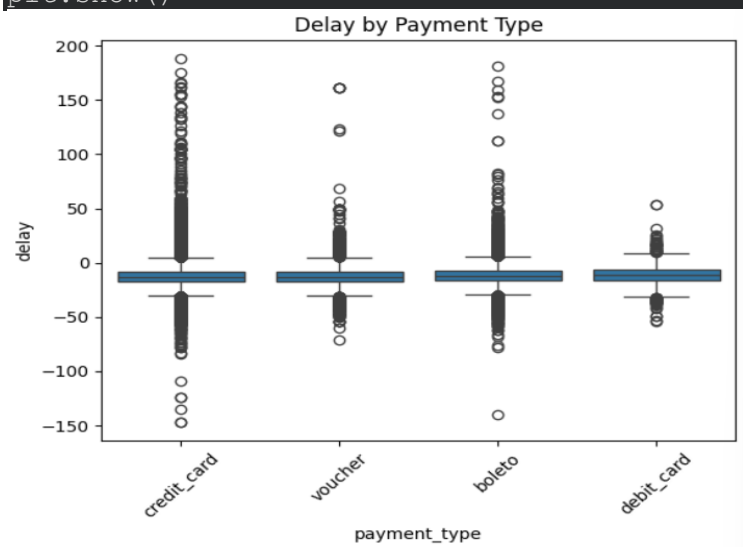


Insights:

- Delivery time varies across months.
- Higher delivery times are observed at the beginning and end of the year.
- Mid-year months show relatively faster delivery times.
- This indicates a possible seasonal effect on delivery performance.

3.Delay vs Payment Type

```
df.groupby('payment_type')['delay'].mean()
sns.boxplot(x='payment_type', y='delay', data=df)
plt.title("Delay by Payment Type")
plt.xticks(rotation=45)
plt.show()
```

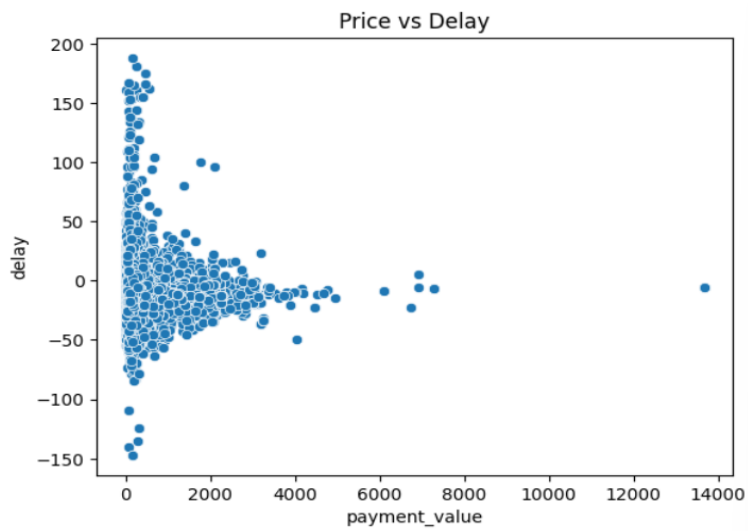


Insights:

- Different payment types show slight variation in delay distribution.
- However, no payment type shows a significantly higher delay compared to others.
- Presence of outliers indicates some extreme cases of delay across all payment methods

4.Payment value vs Delay

```
sns.scatterplot(x='payment_value', y='delay', data=df)
plt.title("Price vs Delay")
plt.show()
```



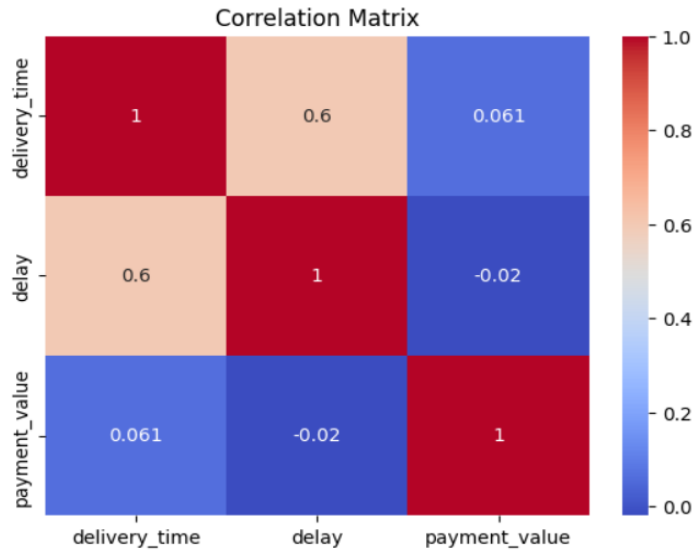
Insights:

- No strong relationship is observed between payment value and delay.
- Most data points are clustered at lower payment values.
- High-value orders do not necessarily result in higher delays.

The bivariate analysis suggests that temporal factors such as month have some influence on delivery time, while factors like payment type and payment value have minimal impact on delays. The variations observed are relatively small, indicating that delays are likely influenced by other operational or logistical factors not captured directly in these variables.

STEP 9: Correlation Analysis

```
df[['delivery_time','delay','payment_value']].corr()
sns.heatmap(df[['delivery_time','delay','payment_value']].corr(),
annot=True, cmap='coolwarm')
plt.title("Correlation Matrix")
plt.show()
```



Insights: Correlation analysis confirms that:-

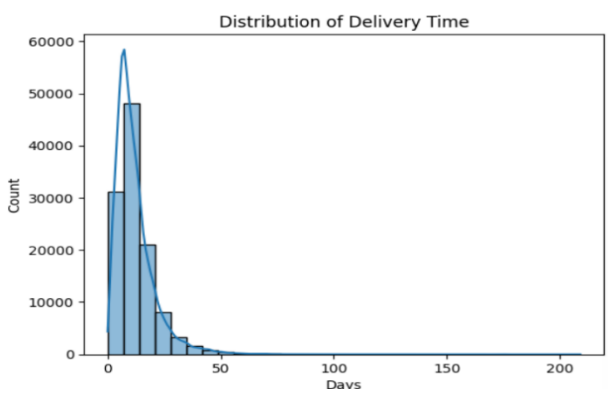
- Delivery-related variables are naturally correlated due to their mathematical relationship.
- Financial factors such as payment value do not significantly impact delivery delays.
- This suggests that delays are more likely influenced by operational factors (such as logistics, location, or shipping efficiency) rather than order value.

NOTE: The correlation between **delivery_time** and **delay** is expected and not a strong insight, since delay is directly calculated using delivery timestamps. Therefore, this relationship does not indicate causation but rather a mathematical dependency.

STEP 10: Data Visualization

1.Distribution of Delivery Time:

```
sns.histplot(df['delivery_time'], bins=30, kde=True)
plt.title("Distribution of Delivery Time")
plt.xlabel("Days")
plt.ylabel("Count")
plt.show()
```



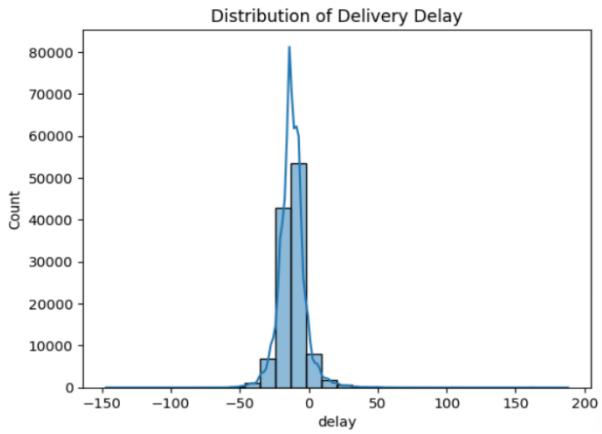
Insights:

- Most orders are delivered within a short time range (approximately 5–20 days).
- The distribution is right-skewed, indicating a few orders take significantly longer time.
- Presence of extreme values suggests occasional delivery inefficiencies.

2.Distribution of Delay

```
sns.histplot(df['delay'], bins=30, kde=True)
```

```
plt.title("Distribution of Delivery Delay")
plt.show()
```

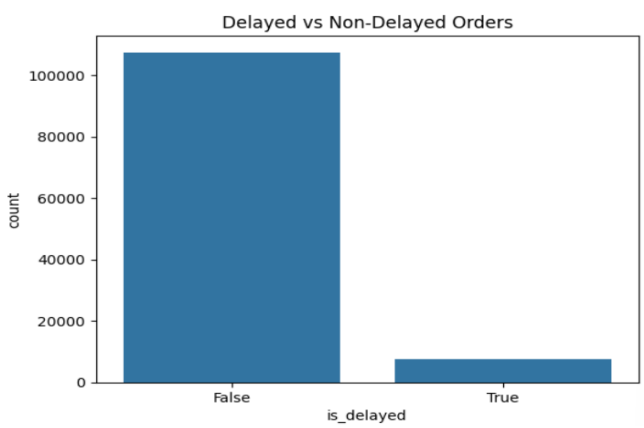


Insights:

- The distribution is centered slightly below zero.
- This indicates that most orders are delivered before the estimated delivery date.
- A small portion of orders experience positive delay (late delivery).

3.Delayed vs Non-Delayed Orders:

```
df['is_delayed'].value_counts(normalize=True) * 100
sns.countplot(x='is_delayed', data=df)
plt.title("Delayed vs Non-Delayed Orders")
plt.show()
```



Insights:

- Approximately **93–94%** of orders are not delayed.
- Around **6–7%** of orders are delayed.
- This shows that overall delivery performance is efficient, but a small fraction of delays still exists.

RESULTS:

The analysis of the e-commerce dataset revealed the following:

- A total of **~99,441 unique orders** were analyzed after merging multiple datasets.
- The final dataset contained **~115,000+ rows**, indicating multiple items/payments per order.
- The **average delivery time** is approximately **12 days**, with most deliveries completed within **5–20 days**.

- The **average delay is negative (~ -12 days)**, meaning most orders are delivered **before the estimated date**.
- Only **~6.5% of orders are delayed**, while the majority (~93.5%) are delivered on time or early.
- **Seasonal variation exists**, with delivery time increasing at the beginning and end of the year.
- **Day of order has minimal impact** on delay.
- **Payment type and payment value show no significant influence** on delivery delay.
- A **moderate correlation (~0.6)** exists between delivery time and delay, but this is due to how delay is calculated.
- A small number of **extreme outliers** exist where delivery delays are very high.

Hence, the project demonstrates that **delivery delays are not driven by pricing or payment** behavior, but are more likely caused by **logistics, supply chain, or regional factors** (which were not deeply explored in this analysis).